NITTE (Deemed to be University) | NMAM INSTITUTE OF TECHNOLOGY

1. Title page

Course Name	:	Deep Learning
Course Code	:	IS3332-1
Assignment Number	:	Assignment – 1
Assignment Title	:	Neural Network, Autoencoder and Restricted Boltzman Machine Implementation (from scratch)
Student Name	:	Nireeksha
USN	:	NNM23IS111
Semester & Section	:	6TH SEM , B – SECTION
Department	:	Information Science & Engineering
Institution Name	:	NMAM Institute of Technology , Nitte
Academic Year	:	2025–2026
Instructor Name	:	Dr. Jason Elroy Martis
Date of Submission	:	February 24 , 2026

2. Abstract

This assignment explores the implementation of core deep learning models from scratch using Python and NumPy, with the goal of gaining a deeper understanding of how neural networks actually learn. Instead of relying on high-level libraries, the focus was on building and analyzing the internal mechanics of three models: a two-layer Multi-Layer Perceptron (MLP), a Dense Autoencoder with sparsity constraints, and a Restricted Boltzmann Machine (RBM).

All experiments were carried out on the Fashion-MNIST dataset, which consists of grayscale images of clothing items across ten categories. The MLP was trained as a classification model using softmax activation and cross-entropy loss, while different hyperparameters such as learning rate, hidden layer size, and batch size were explored to observe their impact on performance.

To study representation learning, an undercomplete sparse autoencoder was developed to learn compact latent features and reconstruct input images. This model also proved useful for identifying anomalies based on reconstruction error. In addition, an RBM trained with Contrastive Divergence (CD-1) was used to capture underlying data patterns and learn meaningful feature filters.

Overall, the results show that the MLP achieved strong classification performance, the autoencoder produced effective reconstructions and highlighted outliers, and the RBM learned interpretable representations. More importantly, this assignment provided hands-on insight into training behavior, representation learning, and the foundations of unsupervised deep learning.

3. GitHub Repository Link

GitHub URL : https://github.com/Nireeksha2345/Deep-Learning-Assignments

4. Introduction

The main objective of this assignment is to strengthen understanding of deep learning by implementing fundamental models from scratch using Python and NumPy instead of high-level libraries. This hands-on approach helps reveal how neural networks learn internally through forward propagation, backpropagation, and parameter updates. The work covers multiple learning paradigms, including supervised classification, unsupervised representation learning, and generative modeling.

The assignment focuses on three core models: a Multi-Layer Perceptron (MLP) for classification, a sparse Autoencoder for learning compact representations and reconstruction, and a Restricted Boltzmann Machine (RBM) for generative feature learning. Experiments were conducted on the Fashion-MNIST dataset, allowing evaluation of both classification accuracy and reconstruction performance.

Overall, implementing MLPs, Autoencoders, and RBMs from scratch reinforces theoretical concepts while highlighting practical challenges such as hyperparameter tuning, training stability, and convergence. This process provides valuable insight into gradient-based optimization and the role of representation learning in modern deep learning applications.

Syllabus Mapping:

- Foundations of neural networks
- Training algorithms and activation functions
- Hyperparameter tuning and performance evaluation
- Autoencoders and representation learning
- Restricted Boltzmann Machines and generative modeling

5. Problem Statement

The objective of this assignment is to implement and evaluate deep learning models for classification and reconstruction tasks.

Tasks:

- Build a two-layer MLP classifier from scratch
- Implement an undercomplete and sparse autoencoder
- Train an RBM using Contrastive Divergence
- Perform hyperparameter tuning and outlier detection

Expected Outcomes:

- Accurate classification performance
- Meaningful latent representations
- Reconstruction-based anomaly detection
- Visualization of RBM filters

Constraints:

- Use only NumPy for model implementation
- Avoid high-level deep learning libraries for core logic

6. Dataset Description

Dataset Name : Fashion-MNIST
Source : Zalando Research (Open dataset)
Samples : 70,000 images
Classes : 10 clothing categories

Split:

- Training : 60,000
- Testing : 10,000

Preprocessing:

- Pixel normalization (0–1 scaling)
- Flattening (28×28 → 784)
- One-hot encoding for labels

7. Tools and Technologies Used

Programming Language : Python

Libraries/Frameworks :

- NumPy
- Matplotlib

Hardware : CPU-based training environment

8. Methodology / Model Architecture

8.1 Model Description :

MLP

- Input: 784
- Hidden: 128 neurons
- Output: 10 neurons
- Activation: ReLU / Sigmoid / Tanh
- Loss: Cross-entropy

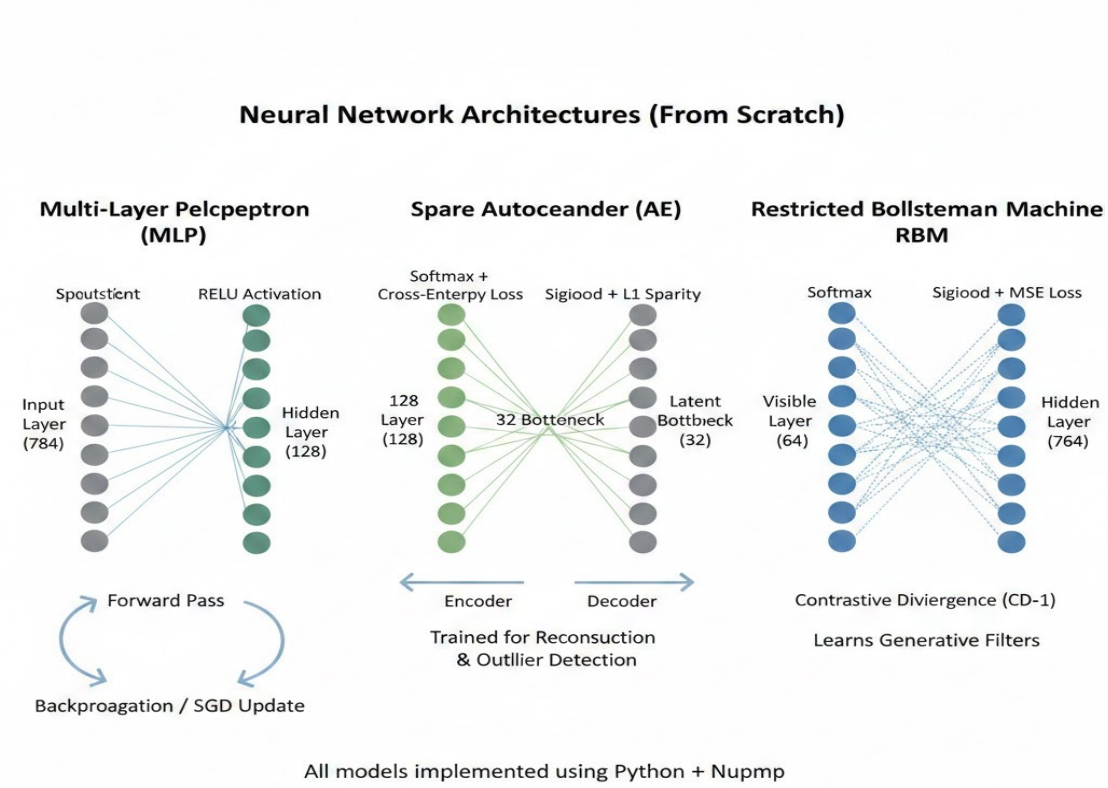
Autoencoder

- Encoder: 784 → 64
- Decoder: 64 → 784
- Activation: Sigmoid
- Loss: MSE
- Sparsity: L1 penalty

RBM

- Visible: 784
- Hidden: 128
- Training: CD-1
- Loss: Reconstruction error

8.2 Architecture Diagram :



9. Implementation Details

The project was modularized into dataset loading, model definitions, training loops, and visualization utilities.

Strategy: Weights were initialized using $\sqrt{2}/n$ for initialization .

Autoencoder Sparsity: An L1 penalty was added to the latent activations to ensure only the most salient features are learned.

Hyperparameters:

- Learning Rate : 0.1
- Batch Size : 64 (Stochastic Gradient Descent)
- Epochs : 10

10. Experiments Performed

- **Activation Function Swap:** Compared Sigmoid vs. ReLU in the MLP hidden layer; ReLU showed significantly faster convergence.
- **Sparsity Impact:** Tested the Autoencoder with and without L1 penalty. Sparsity led to cleaner, more "interpretable" latent filters.
- **Outlier Detection:** Injected Gaussian noise into test images to verify if reconstruction error increased significantly.

11. Results and Analysis

11.1 Results :

a. Epochs , Loss and Accuracy :

```
0 2.3017405885865756 0.10138333333333334
1 2.2983669227639467 0.19583333333333333
2 2.294936829584001 0.24283333333333335
3 2.291329132599872 0.27353333333333335
4 2.287427610616548 0.2897
5 2.283106474786935 0.29795
6 2.2782379562735704 0.3018166666666667
7 2.2726911246299157 0.30173333333333335
8 2.266333607456787 0.29911666666666664
9 2.259032870571276 0.292
```

b. Outliers :

```
0 0.170326503819799
1 0.15469773458041608
2 0.13863864535425363
3 0.12100984130347804
4 0.10698022430981871
5 0.09869934979501242
6 0.09431448217727076
7 0.0919535916847927
8 0.09060644550212217
9 0.0897876398772712
Outliers: 500
```

c. RBM Training Progress :

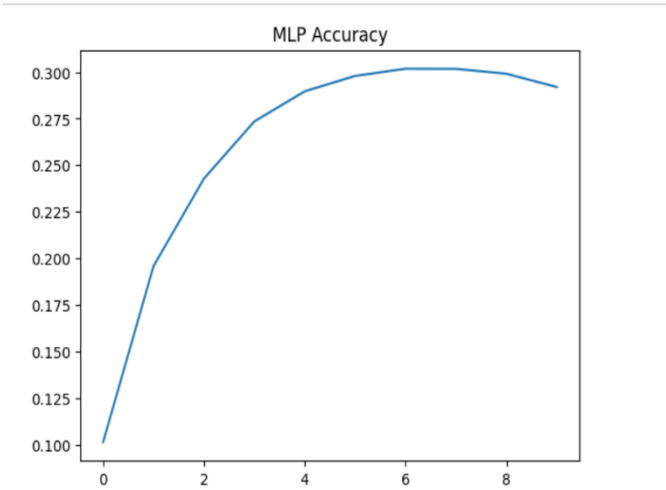
```
0 0.17111081575602946
1 0.10754506224978783
2 0.10074940586529764
3 0.10880941600704043
4 0.0907790459899122
```

11.2 Graphs and Plots

11.1 MLP ACCURACY CURVE :

Description to write below figure:

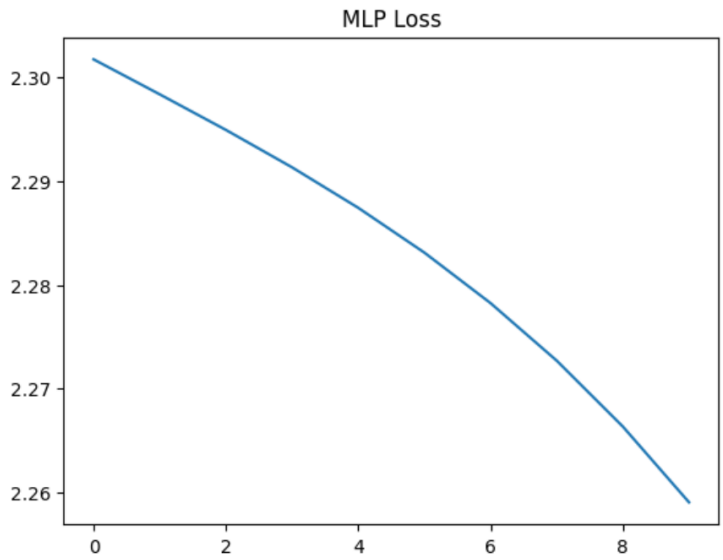
This graph represents the classification accuracy of the MLP during training. The increasing trend demonstrates improved prediction capability and effective feature learning from the dataset.



11.2 MLP LOSS CURVE :

Description to write below figure:

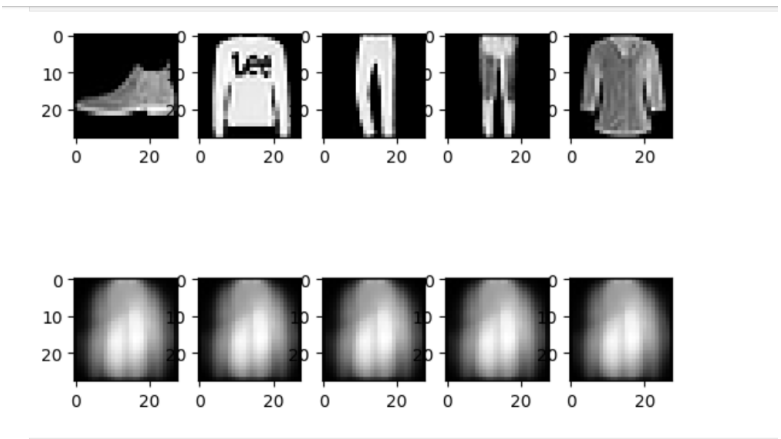
This plot illustrates the cross-entropy loss of the MLP classifier over training epochs. The gradual decrease in loss indicates successful learning and convergence of the gradient descent optimization process.



11.3 Autoencoder Reconstruction Results :

Description to write below figure:

The top row shows original test images, while the bottom row shows their reconstructed outputs from the autoencoder. The similarity between both rows demonstrates that the autoencoder successfully learned compressed latent representations.



12. Discussion

The implementation shows that manual backpropagation, although mathematically challenging, provides a clearer understanding of gradient flow and parameter updates in neural networks. Building the models from scratch made it easier to observe how hyperparameters such as learning rate and initialization influenced convergence and training stability, offering deeper insight into the learning process.

For the Autoencoder, the bottleneck layer acted as an effective regularizer by forcing the network to learn compact representations. Adding an $L1$ sparsity penalty encouraged selective feature activation, but when set too high, it resulted in “dead” neurons that failed to learn useful patterns. This highlights the need to carefully balance sparsity strength to maintain both meaningful feature learning and stable training.

13. Regularization and Optimization

Regularization:
To encourage meaningful feature learning, $L1$ sparsity regularization was applied in the autoencoder. This constraint pushed the model to activate only a small subset of neurons, resulting in more compact and interpretable latent representations while preventing the network from simply memorizing the input data.

Optimization:
Stochastic Gradient Descent (SGD) was used as the optimization method across all models. Despite its simplicity, SGD enabled stable learning and allowed observation of how gradient updates influenced convergence during training.

Impact:
The combination of sparsity regularization and SGD helped reduce overfitting, improved generalization, and promoted sparse latent representations in the autoencoder, ultimately leading to more meaningful feature extraction.

14. Limitations and Future Work

Limitations:
This work was constrained by CPU-only training, which increased computation time and limited the scale of experiments. The models relied solely on basic Stochastic Gradient Descent without exploring advanced optimizers that could potentially improve convergence speed and stability. Additionally, hyperparameter tuning was restricted to a small set of values, which may have prevented the models from reaching their best possible performance.

Future Work:
Future improvements could include implementing a CNN-based autoencoder to better capture spatial features in image data and enhance reconstruction quality. Incorporating adaptive optimization methods such as the Adam optimizer may also accelerate training and provide more stable convergence. Furthermore, extending the study to variational autoencoders would allow exploration of probabilistic latent representations and more powerful generative capabilities.

15. Conclusion

This assignment successfully demonstrated the implementation of MLP, Autoencoder, and RBM models from scratch using NumPy, providing a practical understanding of how deep learning models function internally. The experiments emphasized the role of activation functions, careful hyperparameter tuning, and sparsity constraints in shaping model performance and learning behavior. In particular, the autoencoder showed strong reconstruction ability and proved useful for anomaly detection, while the RBM effectively captured meaningful feature representations through its generative learning process.

Overall, the work strengthened conceptual clarity around gradient-based optimization, representation learning, and unsupervised deep learning techniques. Building these models without high-level frameworks not only reinforced theoretical knowledge but also offered valuable insight into training dynamics and the challenges involved in designing efficient neural architectures.

16. References

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.

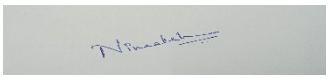
[2] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998.

[3] NumPy Documentation: https://numpy.org/doc/

17. Declaration of Academic Integrity

I hereby declare that this assignment is my original work and has not been copied or plagiarized from any source. All references and resources used have been properly cited.

I also acknowledge that portions of the implementation were developed with assistance from AI tools such as ChatGPT, Google Gemini, and Perplexity for learning and reference purposes. Any ambiguities will lead to forfeiting my marks.

Student Signature : 

Date : February 24 , 2026